CRM Module — Complete API Reference

> ***Base URL:*** `https://<your-domain>/api` ***Authentication:*** *All endpoints (except Public Lead endpoints) require a* ***JWT Bearer token****. Add the header:* `Authorization: Bearer <token>`
> ***Multi-Tenant:*** *The tenant is resolved from the JWT token. All data is automatically scoped to the authenticated tenant.*

Table of Contents

1. Dashboard
2. Leads
3. Companies
4. Contacts
5. Deals
6. Events
7. Tasks
8. Todos
9. Call Logs
10. Emails
11. Calendar & Reminders
12. Lead Stages (Settings)
13. Task Stages (Settings)
14. Lead Sources (Settings)
15. Industries (Settings)
16. Company Types (Settings)
17. Products (CRM Catalog)
18. Sales Products
19. KPIs
20. KPI Ranges
21. Reports
22. Roles & Permissions
23. Permissions (CRUD)
24. Lead Attachments
25. Public Lead Intake
26. Lead Integration (API Key)
27. Enum Reference
28. Common Error Responses

---

1. Dashboard

GET `/api/crm/dashboard/summary`

Get dashboard statistics and recent activities.

Permission: `canView('DASHBOARD')`

Request: No body required.

Response `200 OK`:

```
{
  "stats": {
    "totalCompanies": 42,
    "totalContacts": 128,
    "openDeals": 15,
    "totalRevenue": 250000.00,
    "pendingTasks": 8
  },
  "recentActivities": [
    {
      "id": 1,
      "type": "LEAD",
      "description": "New lead John Doe created",
      "createdAt": "2026-06-01T10:30:00+05:30"
    }
  ]
}
```

---

2. Leads

Base path: `/api/crm/leads`

POST `/api/crm/leads`

Create a new lead.

Permission: `canCreate('LEAD')`

Request Body:

```
{
  "firstName": "John",              // REQUIRED
  "lastName": "Doe",                // REQUIRED
  "leadNo": "LEAD-001",             // REQUIRED (unique lead number)
  "companyId": 1,                   // REQUIRED (CRM Company ID)
  "industryId": 3,                  // optional
  "designation": "Manager",         // optional
  "phone": "+91-9876543210",        // optional
  "email": "john@example.com",      // optional, must be valid email
  "website": "https://example.com",// optional
  "productIds": [1, 2, 3],          // optional, Set<Long>
  "requirements": "Need 500 units",// optional
  "leadSourceId": 2,                // optional
  "address": {                      // optional
    "street": "123 Main St",
    "city": "Mumbai",
    "state": "Maharashtra",
    "zip": "400001",
    "country": "India"
  },
  "notes": "Contacted via trade show",
```

```
  "ownerId": 10,                  // optional, Employee ID
  "currentStageId": 1,            // optional, Lead Stage ID
  "locationId": 5,                // optional
  "forecastCategory": "PIPELINE",  // optional, see Enum Reference
  "expectedCloseDate": "2026-12-31",// optional, format: yyyy-MM-dd
  "amount": 150000.00,            // optional
  "status": "NEW"                 // optional, see Enum Reference
}
```

Response `201 Created` : Returns `CrmLeadResponse` (see GET /{id} response)

GET `/api/crm/leads`

Get paginated list of leads with optional filters.

Permission: `canView('LEAD')`

Query Parameters:

Parameter	Type	Required	Description
`q`	String	No	Search keyword (name, email, phone, etc.)
`status`	String	No	Filter by status (e.g., `NEW`, `ACTIVE`)
`ownerId`	Long	No	Filter by owner employee ID
`period`	String	No	Filter by period (e.g., `TODAY`, `THIS_WEEK`)
`page`	Int	No	Page number (default: 0)
`size`	Int	No	Page size (default: 10)
`sort`	String	No	Sort field (default: `createdAt,desc`)

Response `200 OK` (Paginated):

```
{
  "content": [
    {
      "id": 1,
      "firstName": "John",
      "lastName": "Doe",
      "leadNo": "LEAD-001",
      "companyId": 1,
      "companyName": "Acme Corp",
      "industryId": 3,
      "industryName": "Technology",
      "designation": "Manager",
      "phone": "+91-9876543210",
      "email": "john@example.com",
      "website": "https://example.com",
      "products": [
        { "id": 1, "name": "Product A" }
```

```
      ],
      "requirements": "Need 500 units",
      "leadSourceId": 2,
      "leadSourceName": "Website",
      "address": {
        "street": "123 Main St",
        "city": "Mumbai",
        "state": "Maharashtra",
        "zip": "400001",
        "country": "India"
      },
      "notes": "Contacted via trade show",
      "ownerId": 10,
      "ownerName": "Jane Smith",
      "currentStageId": 1,
      "currentStageName": "Prospecting",
      "locationId": 5,
      "locationName": "HQ",
      "forecastCategory": "PIPELINE",
      "expectedCloseDate": "2026-12-31",
      "amount": 150000.00,
      "status": "NEW",
      "createdAt": "2026-06-01T10:30:00+05:30",
      "updatedAt": "2026-06-01T10:30:00+05:30"
    }
  ],
  "totalElements": 100,
  "totalPages": 10,
  "size": 10,
  "number": 0
}
```

GET `/api/crm/leads/{id}`

Get a single lead by ID.

Permission: `canView('LEAD')`

Path Parameter: `id` — Long, the lead ID

Response `200 OK`: Same as single object in the list above.

Error `404 Not Found`:


```
{ "message": "Lead not found with id: 99" }
```

PUT `/api/crm/leads/{id}`

Update an existing lead.

Permission: `canUpdate('LEAD')`

Path Parameter: `id` — Long, the lead ID

Request Body: Same as POST (all fields optional, only changed fields need to be sent)

Response `200 OK` **:** Updated `CrmLeadResponse`

DELETE `/api/crm/leads/{id}`

Delete a lead.

Permission: `canDelete('LEAD')`

Path Parameter: `id` — Long, the lead ID

Response `204 No Content`

PUT `/api/crm/leads/{id}/status`

Update only the status of a lead.

Permission: `canUpdate('LEAD')`

Request Body:

```
{
  "status": "ACTIVE"   // REQUIRED, see Enum Reference for CrmLeadStatus values
}
```

Response `200 OK` **:** Updated `CrmLeadResponse`

PUT `/api/crm/leads/{id}/stage`

Move a lead to a different stage.

Permission: `canUpdate('LEAD')`

Request Body:

```
{
  "stageId": 3   // REQUIRED, Lead Stage ID
}
```

Response `200 OK` **:** Updated `CrmLeadResponse`

GET `/api/crm/leads/export`

Export leads as CSV file.

Permission: `canExport('LEAD')`

Query Parameters:

Parameter	Type	Required	Description

`ids`	List<Long>	YES	Comma-separated list of lead IDs

Example: `/api/crm/leads/export?ids=1,2,3,4`

Response `200 OK`: Returns a downloadable CSV file.

- Content-Type: `text/csv`

GET `/api/crm/leads/bulk-template`

Download an Excel template for bulk lead upload.

Permission: `canImport('LEAD')`

Response `200 OK`: Downloadable `.xlsx` file.

POST `/api/crm/leads/bulk-upload`

Bulk import leads from an Excel file.

Permission: `canImport('LEAD')`

Content-Type: `multipart/form-data`

Form Data:

Field	Type	Required	Description
`file`	File	YES	Excel (.xlsx) file

Response `200 OK`:

```
{
  "totalRows": 50,
  "successCount": 48,
  "failedCount": 2,
  "errors": [
    {
      "row": 5,
      "leadNo": "LEAD-005",
      "reason": "Company not found"
    }
  ]
}
```

---

3. Companies

Base path: `/api/crm/companies`

POST `/api/crm/companies`

Create a new company.

Permission: `canCreate('COMPANY')`

Request Body:

```
{
  "name": "Acme Corp",              // REQUIRED
  "locationId": 1,                  // optional
  "companyTypeId": 2,               // optional
  "industryId": 3,                  // optional
  "companyOwner": "Jane Smith",     // optional (owner name as string)
  "parentCompanyId": null,          // optional
  "phone": "+91-9876543210",        // optional
  "email": "contact@acme.com",      // optional, must be valid email
  "website": "https://acme.com",    // optional
  "billingStreet": "123 Main St",   // optional
  "billingCity": "Mumbai",          // optional
  "billingZip": "400001",           // optional
  "billingState": "Maharashtra",    // optional
  "billingCountry": "India",        // optional
  "shippingStreet": "456 Ship Ave", // optional
  "shippingCity": "Delhi",          // optional
  "shippingZip": "110001",          // optional
  "shippingState": "Delhi",         // optional
  "shippingCountry": "India"        // optional
}
```

Response `201 Created`:

```
{
  "id": 1,
  "name": "Acme Corp",
  "locationId": 1,
  "locationName": "HQ",
  "companyTypeId": 2,
  "companyTypeName": "Partner",
  "industryId": 3,
  "industryName": "Technology",
  "phone": "+91-9876543210",
  "email": "contact@acme.com",
  "website": "https://acme.com",
  "companyOwner": "Jane Smith",
  "parentCompanyId": null,
  "parentCompanyName": null,
  "billingStreet": "123 Main St",
  "billingCity": "Mumbai",
  "billingZip": "400001",
  "billingState": "Maharashtra",
  "billingCountry": "India",
  "shippingStreet": "456 Ship Ave",
  "shippingCity": "Delhi",
  "shippingZip": "110001",
```

```
  "shippingState": "Delhi",
  "shippingCountry": "India",
  "createdAt": "2026-06-01T10:00:00+05:30",
  "updatedAt": "2026-06-01T10:00:00+05:30"
}
```

GET `/api/crm/companies`

Get all companies (no pagination).

Permission: `canView('COMPANY')`

Response `200 OK`: Array of `CrmCompanyResponse` objects (same structure as above).

GET `/api/crm/companies/{id}`

Get a single company by ID.

Permission: `canView('COMPANY')`

Response `200 OK`: Single `CrmCompanyResponse`

PUT `/api/crm/companies/{id}`

Update a company.

Permission: `canUpdate('COMPANY')`

Request Body: Same as POST

Response `200 OK`: Updated `CrmCompanyResponse`

DELETE `/api/crm/companies/{id}`

Delete a company.

Permission: `canDelete('COMPANY')`

Response `204 No Content`

POST `/api/crm/companies/bulk`

Bulk import companies from an Excel file.

Permission: `canImport('COMPANY')`

Content-Type: `multipart/form-data`

Form Data:

Field	Type	Required	Description
`file`	File	YES	Excel (.xlsx) file

Response `200 OK`: `"Companies imported successfully."`

Response `400 Bad Request`: Array of error strings

```
["Row 3: Company name is required", "Row 7: Invalid email format"]
```

GET `/api/crm/companies/bulk-template`

Download Excel template for bulk company upload.

Permission: `canImport('COMPANY')`

Response `200 OK`: Downloadable `.xlsx` file.

---

4. Contacts

Base path: `/api/contacts`

GET `/api/contacts`

Get paginated list of all contacts.

Permission: `canView('CONTACT')`

Query Parameters:

Parameter	Type	Required	Default	Description
`page`	Int	No	0	Page number
`size`	Int	No	10	Page size

Response `200 OK` (Paginated):

```
{
  "content": [
    {
      "id": 1,
      "leadId": 5,
      "firstName": "Alice",
      "lastName": "Johnson",
      "email": "alice@example.com",
      "phone": "+91-9123456789",
      "createdAt": "2026-06-01T09:00:00+05:30",
      "updatedAt": "2026-06-01T09:00:00+05:30"
    }
  ],
  "totalElements": 50,
  "totalPages": 5,
  "size": 10,
  "number": 0
}
```

GET `/api/contacts/{id}`

Get a single contact by ID.

Permission: `canView('CONTACT')`

Response `200 OK`: Single `ContactDto`

GET `/api/contacts/by-lead/{leadId}`

Get all contacts linked to a specific lead.

Permission: `canView('CONTACT')`

Response `200 OK`: Array of `ContactDto`

POST `/api/contacts`

Create a new contact.

Permission: `canCreate('CONTACT')`

Request Body:

```
{
  "firstName": "Alice",
  "lastName": "Johnson",
  "email": "alice@example.com",
  "phone": "+91-9123456789"
}
```

Response `201 Created`: Created `ContactDto`

PUT `/api/contacts/{id}`

Update a contact.

Permission: `canUpdate('CONTACT')`

Request Body: Same as POST

Response `200 OK`: Updated `ContactDto`

DELETE `/api/contacts/{id}`

Delete a contact.

Permission: `canDelete('CONTACT')`

Response `204 No Content`

---

5. Deals

Base path: `/api/crm/deals`

> *Deals are based on Sales Quotations linked to the current tenant.*

GET `/api/crm/deals`

Get paginated list of deals (quotations).

Permission: `canView('DEAL')`

Query Parameters:

Parameter	Type	Required	Default	Description
`page`	Int	No	0	Page number
`size`	Int	No	1000	Page size

Response `200 OK` (Paginated):

```
{
  "content": [
    {
      "id": 1,
      "quotationNumber": "QUO-2026-001",
      "customerName": "Acme Corp",
      "salespersonName": "Jane Smith",
      "netTotal": 75000.00,
      "status": "CONFIRMED",
      "createdAt": "2026-05-15T08:00:00+05:30"
    }
  ],
  "totalElements": 30,
  "totalPages": 1,
  "size": 1000,
  "number": 0
}
```

> ***Note:*** *`status` values come from Sales module: `DRAFT`, `CONFIRMED`, `CANCELLED`, `DELIVERED`, etc.*

---

6. Events

Base path: `/api/crm/events`

GET `/api/crm/events`

Get all events for the current tenant.

Permission: `canView('EVENT')`

Response `200 OK`: Array of `CrmEventResponse`

```
[
  {
```

```
    "id": 1,
    "tenantId": 5,
    "subject": "MEETING",
    "description": "Quarterly review meeting",
    "sameStartEnd": false,
    "date": "2026-06-15",
    "fromTime": "10:00:00",
    "toTime": "11:00:00",
    "primaryContactId": 3,
    "primaryContactName": "Alice Johnson",
    "employeeIds": [10, 11],
    "employeeNames": ["Jane Smith", "Bob Brown"],
    "contactIds": [3, 4],
    "contactNames": ["Alice Johnson", "Charlie White"],
    "leadId": 5,
    "status": "OPEN",
    "priority": "HIGH",
    "meetingType": "ONLINE",
    "meetingWith": "Client"
  }
]
```

GET `/api/crm/events/{id}`

Get a single event by ID.

Permission: `canView('EVENT')`

Response `200 OK`: Single `CrmEventResponse`

GET `/api/crm/events/lead/{leadId}`

Get all events linked to a lead.

Permission: `canView('EVENT')`

Response `200 OK`: Array of `CrmEventResponse`

POST `/api/crm/events`

Create a new event.

Permission: `canCreate('EVENT')`

Request Body:

```
{
  "subject": "MEETING",           // REQUIRED, Enum: CALL | MEETING | EVENT | TASK |
OTHER
  "description": "Review meeting",// optional
  "sameStartEnd": false,          // REQUIRED (true = single time point)
  "date": "2026-06-15",           // REQUIRED, format: yyyy-MM-dd
  "fromTime": "10:00",            // REQUIRED, format: HH:mm
```

```
  "toTime": "11:00",              // optional (if sameStartEnd = false)
  "primaryContactId": 3,          // optional
  "employeeIds": [10, 11],        // optional, List<Long>
  "contactIds": [3, 4],           // optional, List<Long>
  "status": "OPEN",               // REQUIRED, see Enum Reference
  "priority": "HIGH",             // REQUIRED, Enum: LOW | NORMAL | HIGH
  "leadId": 5,                    // optional
  "meetingType": "ONLINE",        // optional, Enum: ONLINE | ONSITE
  "meetingWith": "Client"         // optional
}
```

Response `200 OK` : Created `CrmEventResponse`

PUT `/api/crm/events/{id}`

Update an event.

Permission: `canUpdate('EVENT')`

Request Body: Same as POST

Response `200 OK` : Updated `CrmEventResponse`

DELETE `/api/crm/events/{id}`

Delete an event.

Permission: `canDelete('EVENT')`

Response `204 No Content`

POST `/api/crm/events/availability/{employeeId}`

Check if an employee is busy during a given event time.

Permission: `canView('EVENT')`

Path Parameter: `employeeId` — Long

Request Body: Same as POST Event (uses `date` , `fromTime` , `toTime`)

Response `200 OK` :

```
true    // true = BUSY (conflict exists)
false   // false = FREE
```

---

7. Tasks

Base path: `/api/crm/tasks`

GET `/api/crm/tasks`

Get all tasks for the current tenant.

Permission: `canView('TASK')`

Response `200 OK`: Array of `CrmTaskResponse`


```
[
  {
    "id": 1,
    "tenantId": 5,
    "subject": "CALL",
    "comments": "Follow up on proposal",
    "dueDate": "2026-06-10",
    "callTime": "14:00:00",
    "assignedToId": 10,
    "assignedToName": "Jane Smith",
    "employees": [
      { "id": 10, "name": "Jane Smith" },
      { "id": 11, "name": "Bob Brown" }
    ],
    "contacts": [
      { "id": 3, "name": "Alice Johnson" }
    ],
    "leadId": 5,
    "status": "OPEN",
    "createdAt": "2026-06-01T10:00:00+05:30",
    "updatedAt": "2026-06-01T10:00:00+05:30"
  }
]
```

GET `/api/crm/tasks/{id}`

Get a single task by ID.

Permission: `canView('TASK')`

Response `200 OK`: Single `CrmTaskResponse`

GET `/api/crm/tasks/lead/{leadId}`

Get all tasks linked to a specific lead.

Permission: `canView('TASK')`

Response `200 OK`: Array of `CrmTaskResponse`

POST `/api/crm/tasks`

Create a new task.

Permission: `canCreate('TASK')`

Request Body:

```
{
  "subject": "CALL",              // REQUIRED, Enum: CALL | EVENT | MEETING | OTHER
| TASK
  "comments": "Follow up call",  // optional
  "dueDate": "2026-06-10",       // optional, format: yyyy-MM-dd
  "callTime": "14:00",           // optional, format: HH:mm
  "assignedToEmployeeId": 10,    // optional (primary assignee)
  "employeeIds": [10, 11],       // optional, Set<Long> (participants)
  "contactIds": [3],             // optional, Set<Long>
  "leadId": 5,                   // optional
  "status": "OPEN"               // optional (e.g., "OPEN", "DONE")
}
```

Response `201 Created` : Created `CrmTaskResponse`

PUT `/api/crm/tasks/{id}`

Update a task.

Permission: `canUpdate('TASK')`

Request Body: Same as POST

Response `200 OK` : Updated `CrmTaskResponse`

DELETE `/api/crm/tasks/{id}`

Delete a task.

Permission: `canDelete('TASK')`

Response `204 No Content`

---

8. Todos

Base path: `/api/crm/todos`

POST `/api/crm/todos`

Create a new todo.

Permission: `canCreate('TODO')`

Request Body:

```
{
  "subject": "CALL",                // optional, Enum: CALL | EVENT | MEETING |
OTHER | TASK
  "description": "Schedule demo",   // optional
  "dueDate": "2026-06-20",          // optional, format: yyyy-MM-dd
  "fromTime": "09:00",              // optional, format: HH:mm
  "toTime": "10:00",                // optional, format: HH:mm
  "status": "OPEN",                 // optional, see TodoStatus enum
```

```
  "priority": "HIGH",                // optional, Enum: LOW | NORMAL | HIGH
  "customerContactName": "John Doe", // optional
  "assignedToEmployeeId": 10,        // optional
  "employeeIds": [10, 11],           // optional, Set<Long>
  "contactIds": [3],                 // optional, Set<Long>
  "labelIds": [1, 2]                 // optional, Set<Long>
}
```

Response `200 OK` : Created `CrmTodoResponse`

```
{
  "id": 1,
  "subject": "CALL",
  "description": "Schedule demo",
  "dueDate": "2026-06-20",
  "fromTime": "09:00:00",
  "toTime": "10:00:00",
  "status": "OPEN",
  "priority": "HIGH",
  "customerContactName": "John Doe",
  "assignedToId": 10,
  "assignedToName": "Jane Smith",
  "employees": [ { "id": 10, "name": "Jane Smith" } ],
  "contacts": [ { "id": 3, "name": "Alice Johnson" } ],
  "labels": [ { "id": 1, "name": "Urgent" } ],
  "createdAt": "2026-06-01T10:00:00+05:30",
  "updatedAt": "2026-06-01T10:00:00+05:30"
}
```

GET `/api/crm/todos/{id}`

Get a single todo by ID.

Permission: `canView('TODO')`

Response `200 OK` : Single `CrmTodoResponse`

PUT `/api/crm/todos/{id}`

Update a todo.

Permission: `canUpdate('TODO')`

Request Body: Same as POST

Response `200 OK` : Updated `CrmTodoResponse`

DELETE `/api/crm/todos/{id}`

Delete a todo.

Permission: `canDelete('TODO')`

Response `204 No Content`

POST `/api/crm/todos/search`

Search/filter todos with pagination.

Permission: `canView('TODO')`

Query Parameters:

Parameter	Type	Required	Default	Description
`page`	Int	No	0	Page number
`size`	Int	No	20	Page size

Request Body:

```
{
  "dateMode": "SHOW_ALL",          // TODAY | NEXT_DAY | NEXT_WEEK | LAST_2_WEEKS |
SHOW_ALL | CUSTOM
  "fromDate": "2026-06-01",        // only for dateMode = CUSTOM
  "toDate": "2026-06-30",          // only for dateMode = CUSTOM
  "priority": "HIGH",              // optional, null = All
  "status": "OPEN",                // optional, null = All
  "subject": "CALL",               // optional, null = All
  "labelId": 1,                    // optional, null = All labels
  "sortBy": "CREATED_DATE",        // DUE_DATE | CREATED_DATE | UPDATED_DATE
  "direction": "DESC",             // ASC | DESC
  "employeeIds": [10, 11]          // optional, filter by participants
}
```

Response `200 OK` **(Paginated):** Page of `CrmTodoResponse`

GET `/api/crm/todos/subject-counts`

Get count of todos grouped by subject.

Permission: `canView('TODO')`

Response `200 OK`**:**

```
[
  { "label": "CALL", "count": 5 },
  { "label": "MEETING", "count": 3 }
]
```

GET `/api/crm/todos/labels`

Get all todo labels.

Permission: `canView('TODO')`

Response `200 OK`:

```
[
  { "id": 1, "name": "Urgent", "color": "#FF0000" }
]
```

POST `/api/crm/todos/labels`

Create a new label.

Permission: `canCreate('TODO')`

Request Body:

```
{
  "name": "Urgent",    // REQUIRED
  "color": "#FF0000"   // optional (hex color)
}
```

Response `200 OK`: Created label object

PUT `/api/crm/todos/labels/{id}`

Update a label.

Permission: `canUpdate('TODO')`

Request Body: Same as POST

Response `200 OK`: Updated label object

DELETE `/api/crm/todos/labels/{id}`

Delete a label.

Permission: `canDelete('TODO')`

Response `204 No Content`

---

9. Call Logs

Base path: `/api/crm/call-logs`

GET `/api/crm/call-logs`

Get paginated list of all call logs.

Permission: `canView('CALL_LOG')`

Query Parameters:

Parameter	Type	Required	Default	Description

`page`	Int	No	0	Page number
`size`	Int	No	10	Page size

Response `200 OK` (Paginated):


```
{
  "content": [
    {
      "id": 1,
      "comments": "Discussed pricing",
      "callDate": "2026-06-01",
      "callTime": "15:30:00",
      "remindMeBefore": true,
      "reminderTime": "15:00:00",
      "employeeId": 10,
      "employeeName": "Jane Smith",
      "leadId": 5,
      "leadName": "John Doe",
      "contactId": 3,
      "contactName": "Alice Johnson",
      "createdAt": "2026-06-01T15:30:00+05:30",
      "updatedAt": "2026-06-01T15:30:00+05:30"
    }
  ],
  "totalElements": 25,
  "totalPages": 3,
  "size": 10,
  "number": 0
}
```

GET `/api/crm/call-logs/{id}`

Get a single call log by ID.

Permission: `canView('CALL_LOG')`

Response `200 OK` : Single `CrmCallLogDto`

GET `/api/crm/call-logs/by-lead/{leadId}`

Get all call logs for a specific lead.

Permission: `canView('CALL_LOG')`

Response `200 OK` : Array of `CrmCallLogDto`

POST `/api/crm/call-logs`

Create a new call log.

Permission: `canCreate('CALL_LOG')`

Request Body:

```
{
  "comments": "Discussed pricing",   // optional
  "callDate": "2026-06-01",          // REQUIRED, format: yyyy-MM-dd
  "callTime": "15:30",               // optional, format: HH:mm
  "remindMeBefore": true,            // optional, default: false
  "reminderTime": "15:00",           // optional, format: HH:mm
  "employeeId": 10,                  // optional
  "leadId": 5,                       // optional
  "contactId": 3                     // optional
}
```

Response `201 Created`: Created `CrmCallLogDto`

PUT `/api/crm/call-logs/{id}`

Update a call log.

Permission: `canUpdate('CALL_LOG')`

Request Body: Same as POST

Response `200 OK`: Updated `CrmCallLogDto`

DELETE `/api/crm/call-logs/{id}`

Delete a call log.

Permission: `canDelete('CALL_LOG')`

Response `204 No Content`

---

10. Emails

Base path: `/api/crm/emails`

POST `/api/crm/emails/send`

Send an email and record it in the CRM.

Permission: `canSendEmail('EMAIL')`

Request Body:

```
{
  "leadId": 5,                    // optional (link to lead)
  "contactId": 3,                 // optional (link to contact)
  "sentByEmployeeId": 10,         // optional (for tracking)
  "subject": "Proposal Follow-up", // REQUIRED
  "body": "Dear John, ...",       // REQUIRED
```

```
  "ccAddress": "cc@example.com"    // optional
}
```

Response `201 Created`:

```
{
  "id": 1,
  "fromAddress": "crm@company.com",
  "toAddress": "john@example.com",
  "ccAddress": "cc@example.com",
  "subject": "Proposal Follow-up",
  "body": "Dear John, ...",
  "status": "SENT",
  "leadId": 5,
  "contactId": 3,
  "sentByEmployeeId": 10,
  "sentByEmployeeName": "Jane Smith",
  "createdAt": "2026-06-01T10:00:00+05:30"
}
```

> ***Note:*** *`status` values: `SENT` | `FAILED` | `DRAFT`*

GET `/api/crm/emails`

Get paginated list of all emails.

Permission: `canView('EMAIL')`

Query Parameters:

Parameter	Type	Required	Default	Description
`page`	Int	No	0	Page number
`size`	Int	No	10	Page size

Response `200 OK` (Paginated): Page of `CrmEmailDto`

GET `/api/crm/emails/by-lead/{leadId}`

Get all emails for a specific lead.

Permission: `canView('EMAIL')`

Response `200 OK`: Array of `CrmEmailDto`

---

11. Calendar & Reminders

Base path: `/api/crm/calendar`

GET `/api/crm/calendar/events`

Get all calendar items (tasks + events) for the current user.

Permission: `canView('EVENT')`

Response `200 OK`:

```
[
  {
    "id": 1,
    "type": "TASK",
    "title": "CALL",
    "description": "Follow up call",
    "date": "2026-06-10",
    "startTime": "14:00:00",
    "endTime": null,
    "status": "OPEN",
    "priority": "HIGH",
    "leadName": "John Doe",
    "reminderId": 5,
    "remindAt": "2026-06-10T13:45:00",
    "reminderDismissed": false
  }
]
```

GET `/api/crm/calendar/due-reminders`

Get all due (non-dismissed) reminders.

Permission: `canView('EVENT')`

Response `200 OK`:

```
[
  {
    "id": 1,
    "entityType": "TASK",
    "entityId": 5,
    "entityTitle": "CALL - John Doe",
    "remindAt": "2026-06-10T13:45:00",
    "dismissed": false,
    "channels": ["EMAIL", "PUSH"],
    "recipient": "jane@company.com"
  }
]
```

POST `/api/crm/calendar/reminders`

Create a new reminder.

Permission: `canCreate('EVENT')`

Request Body:

```
{
  "entityType": "TASK",                    // Enum: TASK | EVENT
  "entityId": 5,                           // ID of the task or event
  "remindAt": "2026-06-10T13:45:00",       // datetime (ISO format, no timezone)
  "channels": ["EMAIL", "PUSH"],           // optional, Set<String>
  "recipient": "jane@company.com"          // optional, email recipient
}
```

Response `200 OK`: Created `CrmReminderResponse`

PUT `/api/crm/calendar/reminders/{id}/dismiss`

Dismiss a reminder.

Permission: `canUpdate('EVENT')`

Response `200 OK`: Empty body

DELETE `/api/crm/calendar/reminders/{id}`

Delete a reminder.

Permission: `canDelete('EVENT')`

Response `200 OK`: Empty body

---

12. Lead Stages (Settings)

Base path: `/api/crm/lead-stages`

GET `/api/crm/lead-stages`

Get all lead stages (ordered by sortOrder).

Permission: `canView('SETTINGS')`

Response `200 OK`:

```
[
  {
    "id": 1,
    "name": "Prospecting",
    "sortOrder": 1,
    "isDefault": true,
    "locationId": null,
    "locationName": null,
    "tenantId": 5,
    "moveTo": null,
    "isConverted": false,
    "isWon": false,
    "isMql": false,
    "isSql": false,
```

```
    "probability": 10.0,
    "assignTo": false
  }
]
```

GET `/api/crm/lead-stages/{id}`

Get a single lead stage by ID.

Permission: `canView('SETTINGS')`

Response `200 OK`: Single `CrmLeadStageResponse`

POST `/api/crm/lead-stages`

Create a new lead stage.

Permission: `canCreate('SETTINGS')`

Request Body:

```
{
  "name": "Negotiation",     // REQUIRED
  "sortOrder": 3,            // optional
  "isDefault": false,        // optional
  "locationId": null,        // optional
  "moveTo": null,            // optional, Enum: NEXT | CONVERTED
  "isConverted": false,      // optional
  "isWon": false,            // optional
  "isMql": false,            // optional
  "isSql": false,            // optional
  "probability": 75.0,       // optional (0–100)
  "assignTo": false          // optional
}
```

Response `200 OK`: Created `CrmLeadStageResponse`

PUT `/api/crm/lead-stages/{id}`

Update a lead stage.

Permission: `canUpdate('SETTINGS')`

Request Body: Same as POST

Response `200 OK`: Updated `CrmLeadStageResponse`

DELETE `/api/crm/lead-stages/{id}`

Delete a lead stage.

Permission: `canDelete('SETTINGS')`

Response `204 No Content`

POST `/api/crm/lead-stages/{id}/move-up`

Move a stage one position up (decrease sortOrder).

Permission: `canUpdate('SETTINGS')`

Response `200 OK`: Updated `CrmLeadStageResponse`

POST `/api/crm/lead-stages/{id}/move-down`

Move a stage one position down (increase sortOrder).

Permission: `canUpdate('SETTINGS')`

Response `200 OK`: Updated `CrmLeadStageResponse`

POST `/api/crm/lead-stages/{id}/set-default`

Set a stage as the default stage.

Permission: `canUpdate('SETTINGS')`

Response `200 OK`: Updated `CrmLeadStageResponse`

---

13. Task Stages (Settings)

Base path: `/api/crm/task-stages`

GET `/api/crm/task-stages`

Get all task stages (ordered).

Permission: `canView('SETTINGS')`

Response `200 OK`:


```
[
  {
    "id": 1,
    "statusName": "Open",
    "completed": false,
    "isDefault": true,
    "sortOrder": 1,
    "locationId": null,
    "locationName": null,
    "tenantId": 5
  }
]
```

GET `/api/crm/task-stages/{id}`

Get a single task stage by ID.

Permission: `canView('SETTINGS')`

POST `/api/crm/task-stages`

Create a task stage.

Permission: `canCreate('SETTINGS')`

Request Body:

```
{
  "statusName": "In Review",   // REQUIRED
  "completed": false,          // optional, marks this stage as completion
  "isDefault": false,          // optional
  "sortOrder": 2,              // optional
  "locationId": null           // optional
}
```

Response `200 OK`: Created `CrmTaskStageResponse`

PUT `/api/crm/task-stages/{id}`

Update a task stage.

Permission: `canUpdate('SETTINGS')`

Request Body: Same as POST

Response `200 OK`: Updated `CrmTaskStageResponse`

DELETE `/api/crm/task-stages/{id}`

Delete a task stage.

Permission: `canDelete('SETTINGS')`

Response `204 No Content`

POST `/api/crm/task-stages/{id}/move-up`

Move task stage one position up.

Permission: `canUpdate('SETTINGS')`

Response `200 OK`: Updated `CrmTaskStageResponse`

POST `/api/crm/task-stages/{id}/move-down`

Move task stage one position down.

Permission: `canUpdate('SETTINGS')`

Response `200 OK`: Updated `CrmTaskStageResponse`

POST `/api/crm/task-stages/{id}/set-default`

Set a task stage as default.

Permission: `canUpdate('SETTINGS')`

Response `200 OK`: Updated `CrmTaskStageResponse`

---

14. Lead Sources (Settings)

Base path: `/api/crm/lead-sources`

GET `/api/crm/lead-sources`

Get all lead sources.

Permission: `canView('SETTINGS')`

Response `200 OK`:

```
[
  {
    "id": 1,
    "name": "Website",
    "tenantId": 5,
    "createdAt": "2026-01-01T00:00:00+05:30"
  }
]
```

POST `/api/crm/lead-sources`

Create a lead source.

Permission: `canCreate('SETTINGS')`

Request Body:

```
{
  "name": "Trade Show"   // REQUIRED
}
```

Response `201 Created`: Created lead source response

PUT `/api/crm/lead-sources/{id}`

Update a lead source.

Permission: `canUpdate('SETTINGS')`

Request Body: Same as POST

Response `200 OK`: Updated lead source response

DELETE `/api/crm/lead-sources/{id}`

Delete a lead source.

Permission: `canDelete('SETTINGS')`

Response `204 No Content`

---

15. Industries (Settings)

Base path: `/api/settings/industries`

> *Note: This uses `/api/settings/` prefix, not `/api/crm/`.*

GET `/api/settings/industries`

Get all industries.

Permission: `canView('SETTINGS')`

Response `200 OK`**:**

```
[
  {
    "id": 1,
    "name": "Technology",
    "description": "IT & Software companies"
  }
]
```

GET `/api/settings/industries/{id}`

Get an industry by ID.

Permission: `canView('SETTINGS')`

POST `/api/settings/industries`

Create an industry.

Permission: `canCreate('SETTINGS')`

Request Body:

```
{
  "name": "Healthcare",          // REQUIRED
  "description": "Medical sector" // optional
}
```

Response `200 OK` **:** Created industry DTO

PUT `/api/settings/industries/{id}`

Update an industry.

Permission: `canUpdate('SETTINGS')`

Request Body: Same as POST

Response `200 OK`: Updated industry DTO

DELETE `/api/settings/industries/{id}`

Delete an industry.

Permission: `canDelete('SETTINGS')`

Response `204 No Content`

---

16. Company Types (Settings)

Base path: `/api/crm/company-types`

GET `/api/crm/company-types`

Get all company types.

Permission: `canView('SETTINGS')`

Response `200 OK`:

```
[
  { "id": 1, "name": "Partner" },
  { "id": 2, "name": "Customer" }
]
```

POST `/api/crm/company-types`

Create a company type.

Permission: `canCreate('SETTINGS')`

Request Body:

```
{
  "name": "Vendor"   // REQUIRED
}
```

Response `200 OK`: Created company type DTO

PUT `/api/crm/company-types/{id}`

Update a company type.

Permission: `canUpdate('SETTINGS')`

Request Body: Same as POST

Response `200 OK` : Updated company type DTO

DELETE `/api/crm/company-types/{id}`

Delete a company type.

Permission: `canDelete('SETTINGS')`

Response `204 No Content`

---

17. Products (CRM Catalog)

Base path: `/api/crm/products`

> *These are CRM-specific product catalog items (separate from Sales Products).*

GET `/api/crm/products`

Get all CRM products, optionally filtered by industry.

Permission: `canView('PRODUCT')`

Query Parameters:

Parameter	Type	Required	Description
`industryId`	Long	No	Filter by industry ID

Response `200 OK` :

```
[
  {
    "id": 1,
    "name": "CRM Software",
    "description": "Full CRM package",
    "industryId": 3,
    "industryName": "Technology"
  }
]
```

GET `/api/crm/products/{id}`

Get a single CRM product by ID.

Permission: `canView('PRODUCT')`

POST `/api/crm/products`

Create a CRM product.

Permission: `canCreate('PRODUCT')`

Request Body:

```
{
  "name": "CRM Software",          // REQUIRED
  "description": "Full CRM",      // optional
  "industryId": 3                 // optional
}
```

Response `200 OK`: Created product response

PUT `/api/crm/products/{id}`

Update a CRM product.

Permission: `canUpdate('PRODUCT')`

Request Body: Same as POST

Response `200 OK`: Updated product response

DELETE `/api/crm/products/{id}`

Delete a CRM product.

Permission: `canDelete('PRODUCT')`

Response `204 No Content`

---

18. Sales Products

Base path: `/api/crm/sales-products`

> *Full product catalog with image upload, barcode generation, and pricing.*

POST `/api/crm/sales-products`

Create a sales product.

Permission: `canCreate('PRODUCT')`

Supports two content types:

Option A — JSON:

```
{
  "itemType": "PRODUCT",           // Enum: PRODUCT | SERVICE | RAW_MATERIAL
  "isPurchase": true,
  "isSales": true,
  "itemCode": "SKU-001",
  "name": "Widget Pro",
  "description": "High quality widget",
  "imageUrl": null,
  "unitOfMeasure": "PCS",
  "reorderLimit": 50,
  "vatClassificationCode": "VAT5",
```

```
  "purchasePrice": 100.00,
  "salesPrice": 150.00,
  "tax": "GST",
  "taxRate": 18.0
}
```

Option B — Multipart (with image):

- Part `product` : JSON as above
- Part `image` (optional): Image file (jpg, png, gif)

Response `200 OK` :

```
{
  "id": 1,
  "itemType": "PRODUCT",
  "isPurchase": true,
  "isSales": true,
  "itemCode": "SKU-001",
  "name": "Widget Pro",
  "description": "High quality widget",
  "imageUrl": "products/images/widget-pro.jpg",
  "unitOfMeasure": "PCS",
  "reorderLimit": 50,
  "vatClassificationCode": "VAT5",
  "purchasePrice": 100.00,
  "salesPrice": 150.00,
  "tax": "GST",
  "barcode": "1234567890123",
  "barcodeUrl": "products/barcodes/sku-001.png",
  "taxRate": 18.0,
  "createdAt": "2026-06-01T10:00:00+05:30",
  "updatedAt": "2026-06-01T10:00:00+05:30"
}
```

GET `/api/crm/sales-products`

Get paginated list of sales products.

Permission: `canView('PRODUCT')`

Query Parameters:

Parameter	Type	Required	Default	Description
`page`	Int	No	0	Page number
`size`	Int	No	10	Page size

Response `200 OK` (Paginated): Page of `CrmSalesProductResponse`

GET `/api/crm/sales-products/{id}`

Get a single sales product by ID.

Permission: `canView('PRODUCT')`

Response `200 OK` **:** Single `CrmSalesProductResponse`

PUT `/api/crm/sales-products/{id}`

Update a sales product. Supports JSON or Multipart (same as POST).

Permission: `canUpdate('PRODUCT')`

Response `200 OK` **:** Updated `CrmSalesProductResponse`

DELETE `/api/crm/sales-products/{id}`

Delete a sales product.

Permission: `canDelete('PRODUCT')`

Response `204 No Content`

GET `/api/crm/sales-products/{id}/image`

Get the product image file.

Permission: `canView('PRODUCT')`

Response `200 OK` **:** Image file (Content-Type: image/jpeg, image/png, etc.)

GET `/api/crm/sales-products/{id}/barcode`

Get the product barcode image.

Permission: `canView('PRODUCT')`

Response `200 OK` **:** Barcode image file

GET `/api/crm/sales-products/images/**`

Serve a product image by file path.

Permission: `canView('PRODUCT')`

Example: `/api/crm/sales-products/images/products/widget-pro.jpg`

Response `200 OK` **:** Image file

---

19. KPIs

Base path: `/api/crm/kpis`

GET `/api/crm/kpis`

Get all KPIs.

Permission: `canView('KPI')`

Response `200 OK`:

```
[
  {
    "id": 1,
    "name": "Monthly Calls",
    "description": "Target number of calls per month",
    "dataType": "NUMBER",
    "type": "MONTHLY",
    "tenantId": 5,
    "locationId": null,
    "locationName": null,
    "ranges": [],
    "assignedEmployees": [
      {
        "employeeId": 10,
        "employeeName": "Jane Smith",
        "targetValue": 100.0
      }
    ]
  }
]
```

GET `/api/crm/kpis/{id}`

Get a single KPI by ID.

Permission: `canView('KPI')`

Response `200 OK`: Single `CrmKpiResponse`

POST `/api/crm/kpis`

Create a KPI.

Permission: `canCreate('KPI')`

Request Body:

```
{
  "name": "Monthly Calls",          // REQUIRED
  "description": "Calls target",  // optional
  "dataType": "NUMBER",             // REQUIRED, Enum: NUMBER | CURRENCY | PERCENTAGE
  "type": "MONTHLY",                // REQUIRED, Enum: DAILY | WEEKLY | MONTHLY |
QUARTERLY | YEARLY | CUSTOM
  "locationId": null,               // optional
  "assignedEmployees": [            // optional
    {
      "employeeId": 10,
      "targetValue": 100.0
```

```
    }
  ]
}
```

Response `200 OK` : Created `CrmKpiResponse`

PUT `/api/crm/kpis/{id}`

Update a KPI.

Permission: `canUpdate('KPI')`

Request Body: Same as POST

Response `200 OK` : Updated `CrmKpiResponse`

DELETE `/api/crm/kpis/{id}`

Delete a KPI.

Permission: `canDelete('KPI')`

Response `204 No Content`

---

20. KPI Ranges

Base path: `/api/crm/kpis/{kpiId}/ranges`

> *Ranges define the color-coded thresholds for a KPI.*

GET `/api/crm/kpis/{kpiId}/ranges`

Get all ranges for a KPI.

Permission: `canView('KPI')`

Response `200 OK` :

```
[
  {
    "id": 1,
    "kpiId": 1,
    "label": "Good",
    "minValue": 80.0,
    "maxValue": 100.0,
    "color": "#00FF00"
  }
]
```

GET `/api/crm/kpis/{kpiId}/ranges/{id}`

Get a single range.

Permission: `canView('KPI')`

POST `/api/crm/kpis/{kpiId}/ranges`

Create a range for a KPI.

Permission: `canCreate('KPI')`

Request Body:

```
{
  "label": "Good",       // optional
  "minValue": 80.0,      // optional
  "maxValue": 100.0,     // optional
  "color": "#00FF00"     // optional
}
```

Response `200 OK`: Created range object

PUT `/api/crm/kpis/{kpiId}/ranges/{id}`

Update a range.

Permission: `canUpdate('KPI')`

Request Body: Same as POST

Response `200 OK`: Updated range object

DELETE `/api/crm/kpis/{kpiId}/ranges/{id}`

Delete a range.

Permission: `canDelete('KPI')`

Response `204 No Content`

---

21. Reports

Base path: `/api/crm/reports`

GET `/api/crm/reports/leads-by-stage`

Get lead count grouped by stage.

Permission: `canView('REPORT')`

Response `200 OK`:

```
[
  { "label": "Prospecting", "count": 25 },
  { "label": "Negotiation", "count": 10 }
]
```

GET `/api/crm/reports/leads-by-owner`

Get lead count grouped by owner.

Permission: `canView('REPORT')`

Response `200 OK`: Same format as above (label = owner name)

GET `/api/crm/reports/leads-by-status`

Get lead count grouped by status.

Permission: `canView('REPORT')`

Response `200 OK`: Same format as above (label = status)

GET `/api/crm/reports/activity-summary`

Get activity summary (calls, emails, tasks per employee).

Permission: `canView('REPORT')`

Response `200 OK`:

```
[
  {
    "employeeId": 10,
    "employeeName": "Jane Smith",
    "totalCalls": 15,
    "totalEmails": 8,
    "totalTasks": 20
  }
]
```

GET `/api/crm/reports/audit-log`

Get audit log with filters. **(Admin only)**

Required Role: `SUPER_ADMIN` or `CRM_ADMIN`

Query Parameters:

Parameter	Type	Required	Description
`action`	String	No	Filter by action (e.g., `CREATE`)
`entityType`	String	No	Filter by entity (e.g., `LEAD`)
`fromDate`	String	No	Start date `yyyy-MM-dd`
`toDate`	String	No	End date `yyyy-MM-dd`
`page`	Int	No	Page number

`size`	Int	No	Page size

Response `200 OK` (Paginated):

```
{
  "content": [
    {
      "id": 1,
      "action": "CREATE",
      "entityType": "LEAD",
      "entityId": 5,
      "performedBy": "jane@company.com",
      "timestamp": "2026-06-01T10:00:00+05:30",
      "details": "Lead LEAD-001 created"
    }
  ],
  "totalElements": 500,
  "totalPages": 50,
  "size": 10,
  "number": 0
}
```

---

22. Roles & Permissions

Base path: `/api/crm/roles`

Required Role (class-level): `SUPER_ADMIN` or `CRM_ADMIN`

GET `/api/crm/roles`

Get all CRM roles.

Response `200 OK`:

```
[
  {
    "id": 1,
    "name": "Sales Manager",
    "description": "Can manage leads and deals",
    "tenantId": 5,
    "clonedFromId": null,
    "permissions": [
      {
        "module": "LEAD",
        "canView": true,
        "canCreate": true,
        "canUpdate": true,
        "canDelete": false,
        "canExport": true,
        "canImport": false,
        "canChangeOwner": true,
```

```
        "canSendEmail": true,
        "ownerOnly": false
      }
    ],
    "userCount": 3
  }
]
```

GET `/api/crm/roles/{id}`

Get a single role by ID.

Response `200 OK`: Single `CrmRoleResponse`

POST `/api/crm/roles`

Create a new role.

Request Body:

```
{
  "name": "Sales Rep",              // REQUIRED
  "description": "Sales representative role",
  "cloneRoleId": 1,                // optional, clone permissions from another role
  "permissions": [                  // optional
    {
      "module": "LEAD",
      "canView": true,
      "canCreate": true,
      "canUpdate": true,
      "canDelete": false,
      "canExport": false,
      "canImport": false,
      "canChangeOwner": false,
      "canSendEmail": true,
      "ownerOnly": true
    }
  ]
}
```

> ***Available Modules:*** *`DASHBOARD`, `LEAD`, `COMPANY`, `CONTACT`, `TASK`, `REMINDER`, `PRODUCT`, `CALL_LOG`, `EVENT`, `REPORT`, `SETTINGS`, `ATTACHMENT`, `EMAIL`, `DEAL`, `KPI`, `TODO`*

Response `200 OK`: Created `CrmRoleResponse`

PUT `/api/crm/roles/{id}`

Update a role.

Request Body: Same as POST

Response `200 OK`: Updated `CrmRoleResponse`

DELETE `/api/crm/roles/{id}`

Delete a role.

Response `204 No Content`

POST `/api/crm/roles/{roleId}/users/{employeeId}`

Assign an employee to a CRM role.

Response `200 OK` : Empty body

DELETE `/api/crm/roles/users/{employeeId}`

Remove a user's CRM role assignment.

Response `204 No Content`

GET `/api/crm/roles/my-permissions`

Get the current user's CRM permissions.

Required: Authenticated (any role)

Response `200 OK` :


```
{
  "permissions": {
    "LEAD": {
      "module": "LEAD",
      "canView": true,
      "canCreate": true,
      "canUpdate": true,
      "canDelete": false,
      "canExport": true,
      "canImport": false,
      "canChangeOwner": false,
      "canSendEmail": true,
      "ownerOnly": false
    },
    "COMPANY": { ... },
    "CONTACT": { ... }
  }
}
```


> ***Note:*** *`SUPER_ADMIN` and `CRM_ADMIN` users will receive full (`true`) permissions for all modules.*

POST `/api/crm/roles/seed-defaults`

Seed default roles for the current tenant.

Required Role: `SUPER_ADMIN` or `CRM_ADMIN`

Response `200 OK` : Empty body

---

23. Permissions (CRUD)

Base path: `/api/crm/permissions`

> *Low-level permission management (usually managed through Roles).*

GET `/api/crm/permissions`

Get all permissions.

Response `200 OK` :

```
[
  {
    "id": 1,
    "module": "LEAD",
    "action": "CREATE"
  }
]
```

GET `/api/crm/permissions/{id}`

Get a single permission by ID.

POST `/api/crm/permissions`

Create a permission entry.

Request Body:

```
{
  "module": "LEAD",
  "action": "CREATE"
}
```

PUT `/api/crm/permissions/{id}`

Update a permission entry.

Request Body: Same as POST

DELETE `/api/crm/permissions/{id}`

Delete a permission entry.

Response `204 No Content`

---

24. Lead Attachments

Base path: `/api/crm/leads`

POST `/api/crm/leads/{leadId}/attachments`

Upload a file attachment for a lead.

Permission: `canCreate('ATTACHMENT')`

Content-Type: `multipart/form-data`

Form Data:

Field	Type	Required	Description
`file`	File	YES	File to upload

Response `200 OK`:

```
{
  "id": 1,
  "leadId": 5,
  "fileName": "proposal.pdf",
  "filePath": "attachments/5/proposal.pdf",
  "fileSize": 102400,
  "contentType": "application/pdf",
  "uploadedAt": "2026-06-01T10:00:00+05:30"
}
```

GET `/api/crm/leads/{leadId}/attachments`

Get all attachments for a lead.

Permission: `canView('ATTACHMENT')`

Response `200 OK`: Array of attachment objects (same structure as above)

GET `/api/crm/leads/attachments/{attachmentId}/preview`

Preview/download an attachment.

Permission: `canView('ATTACHMENT')`

Response `200 OK`: File content with appropriate Content-Type

- Supports: `application/pdf`, `image/jpeg`, `image/png`, `image/gif`, `application/octet-stream`
- Header: `Content-Disposition: inline; filename="proposal.pdf"`

DELETE `/api/crm/leads/attachments/{attachmentId}`

Delete an attachment.

Permission: `canDelete('ATTACHMENT')`

Response `204 No Content`

---

25. Public Lead Intake

Base path: `/api/public/leads`

> ***No JWT required.*** *These endpoints are publicly accessible and use an API Key for tenant identification.*

POST `/api/public/leads/intake`

Submit a lead from any external source (website forms, etc.).

Headers:

Header	Required	Description
`X-Api-Key`	YES	Your tenant API key

Request Body:

```
{
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@example.com",
  "phone": "+91-9876543210",
  "company": "Acme Corp",
  "source": "Website",
  "campaign": "Summer 2026",
  "requirements": "Need CRM software",
  "notes": "Referred by partner",
  "metadata": {
    "utm_source": "google",
    "utm_medium": "cpc"
  }
}
```

Response `200 OK`:

```
{
  "status": "success",
  "message": "Lead captured successfully"
}
```

Error `401 Unauthorized`:

```
"Missing X-Api-Key"
```

or

```
"Invalid API Key"
```

POST `/api/public/leads/facebook`

Facebook Lead Ads webhook receiver.

Query Parameters:

Parameter	Type	Required	Description
`api_key`	String	YES	Your tenant API key

Request Body: Facebook Lead Ads webhook payload (JSON)

Response `200 OK`: Same as generic intake

GET `/api/public/leads/facebook`

Facebook Webhook Verification endpoint.

Query Parameters:

Parameter	Type	Required
`hub.mode`	String	YES
`hub.verify_token`	String	YES
`hub.challenge`	String	YES

Response `200 OK`: Returns the `hub.challenge` value as plain text.

---

26. Lead Integration (API Key)

Base path: `/api/notification/lead-integration`

Required Role: `SUPER_ADMIN` or `POS_ADMIN`

GET `/api/notification/lead-integration/api-key`

Get the current API key for lead integration.

Response `200 OK`:

```
{
  "apiKey": "abc123xyz789..."
}
```

POST `/api/notification/lead-integration/api-key/regenerate`

Regenerate the API key (old key becomes invalid).

Response `200 OK`:

```
{
  "apiKey": "newkey456abc..."
}
```

---

27. Enum Reference

CrmLeadStatus

Value	Description
`NONE`	Default, no status assigned
`NEW`	Newly created lead
`DUPLICATE`	Marked as duplicate
`TRANSFERRED`	Transferred from another owner
`NOT_RESPONDING`	Not responding to outreach
`JUNK`	Junk/spam lead
`LOST`	Lost lead
`UNASSIGNED`	Not yet assigned to an owner
`PENDING`	Pending action
`ACTIVE`	Actively being worked on
`LEAD_TO_OPERATION`	Lead converted to an operation

ForecastCategory

Value	Description
`QUOTA`	Against quota
`COMMIT_FORECAST`	Committed forecast
`BEST_CASE_FORECAST`	Best case scenario
`OPEN_PIPELINE`	Open pipeline
`CLOSED_ONLY`	Closed deals only
`PIPELINE`	General pipeline

EventSubject / TaskSubject

`CALL` | `MEETING` | `EVENT` | `TASK` | `OTHER`

EventStatus

`OPEN` | `IN_PROGRESS` | `WAITING_ON_SOMEONE_ELSE` | `DEFERRED` | `COMPLETED` | `CLOSED`

EventPriority

`LOW` | `NORMAL` | `HIGH`

MeetingType

`ONLINE` | `ONSITE`

TodoStatus

`OPEN` | `COMPLETED` | `IN_PROGRESS` | `WAITING_ON_SOMEONE_ELSE` | `DEFERRED` | `CLOSED`

TodoPriority

`LOW` | `NORMAL` | `HIGH`

CrmEntityType (for Reminders)

`TASK` | `EVENT`

KpiType

`DAILY` | `WEEKLY` | `MONTHLY` | `QUARTERLY` | `YEARLY` | `CUSTOM`

KpidataType

`NUMBER` | `CURRENCY` | `PERCENTAGE`

CrmItemType (Sales Products)

`PRODUCT` | `SERVICE` | `RAW_MATERIAL`

EmailStatus

`SENT` | `FAILED` | `DRAFT`

CrmStageMoveTo (Lead Stages)

`NEXT` | `CONVERTED`

---

28. Common Error Responses

400 Bad Request

Validation errors on request body:

```
{
  "timestamp": "2026-06-01T10:00:00.000+05:30",
  "status": 400,
```

```
  "error": "Bad Request",
  "message": "Validation failed",
  "errors": {
    "firstName": "First name is required",
    "email": "A valid email is required"
  }
}
```

---

401 Unauthorized

Missing or invalid JWT token:

```
{
  "status": 401,
  "error": "Unauthorized",
  "message": "JWT token is expired or invalid"
}
```

---

403 Forbidden

Authenticated but insufficient permissions:

```
{
  "status": 403,
  "error": "Forbidden",
  "message": "Access Denied"
}
```

---

404 Not Found

Resource not found:

```
{
  "status": 404,
  "error": "Not Found",
  "message": "Lead not found with id: 99"
}
```

---

500 Internal Server Error

Unexpected server error:

```
{
  "status": 500,
  "error": "Internal Server Error",
  "message": "An unexpected error occurred"
}
```

Authentication Quick Reference

```
POST /api/auth/login
{
  "email": "user@company.com",
  "password": "secret"
}
→ Returns: { "token": "eyJhbGci..." }

Use token in all protected requests:
Authorization: Bearer eyJhbGci...
```

Document generated from source code — Version: June 2026